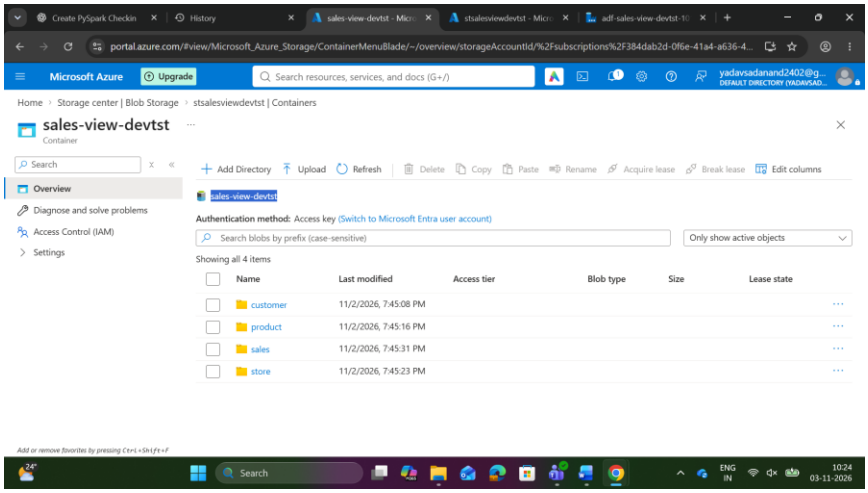
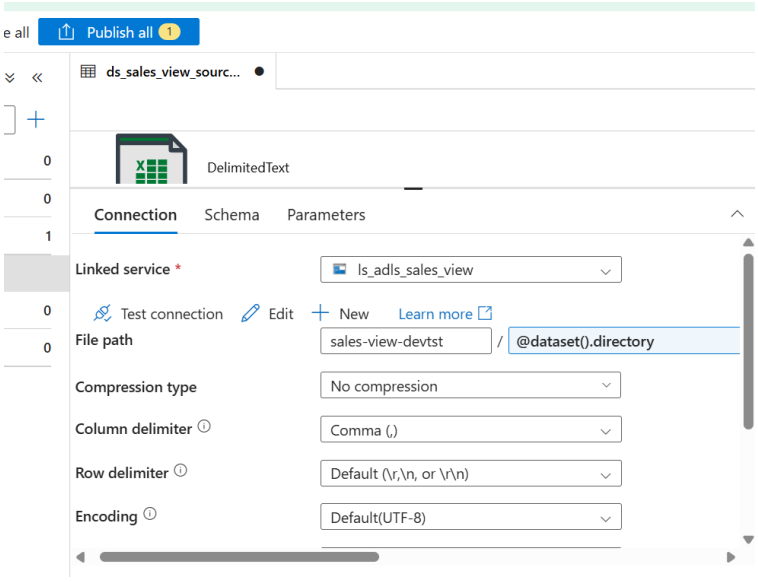
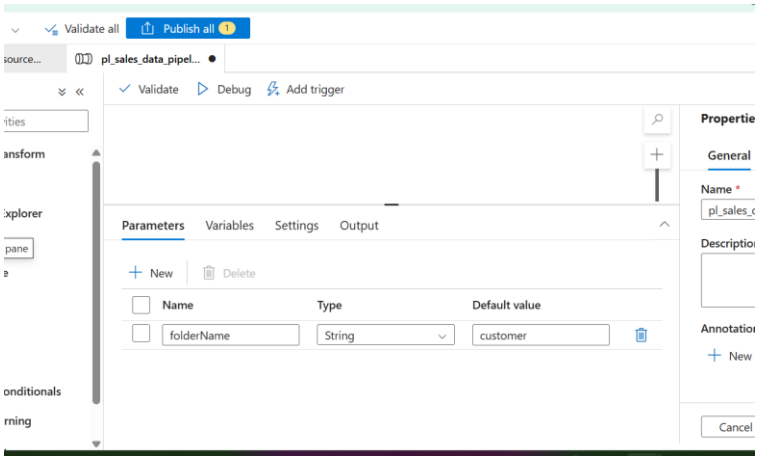
ADF ASSIGNMENT

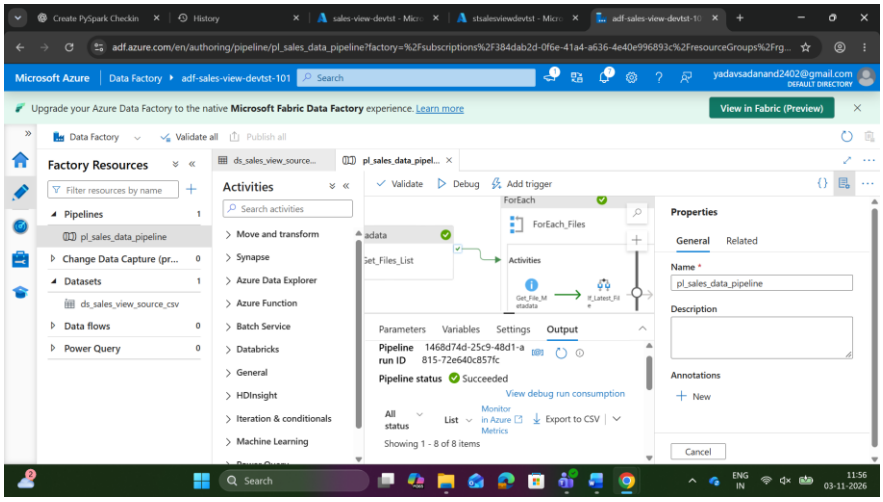
1) Create a container in ADLS with the project name as sales_view_devtst



2) Create an ADF Pipeline to get the latest modified files the folders of the ADLS

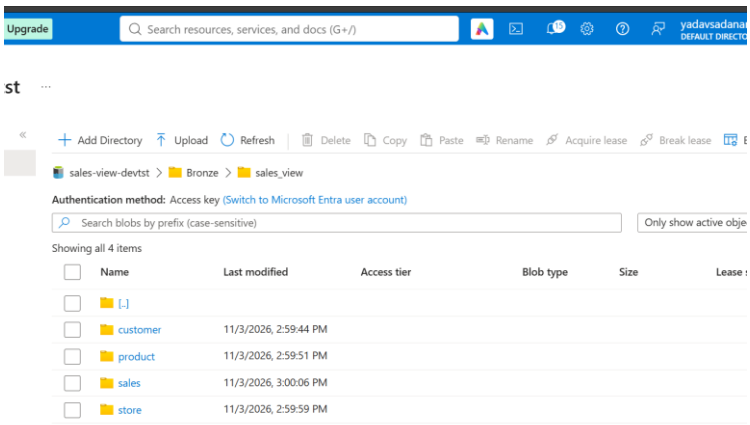






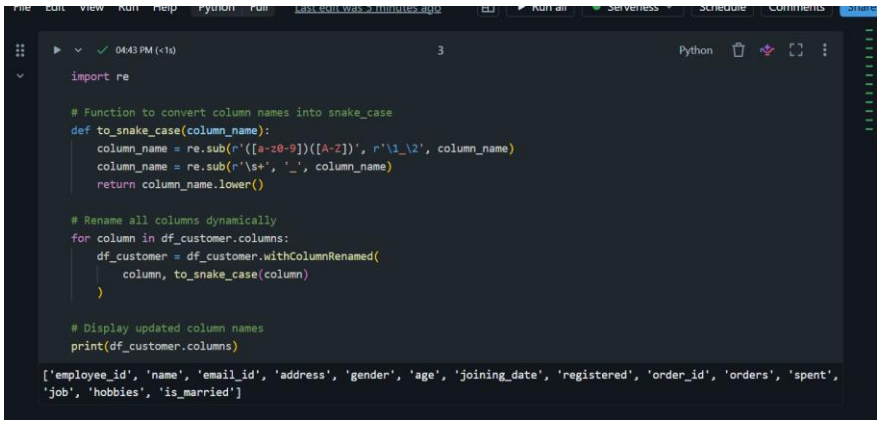
3. Create a Bronze/sales view/ (Bronze – container)

Subfolders-> customer, product, store, sales, and store the raw data copied from the ADF pipeline



4. Customer file

All the column headers should be in snake case in lower case (By using UDF function dynamically works for all camel case to snake case)



```
import re

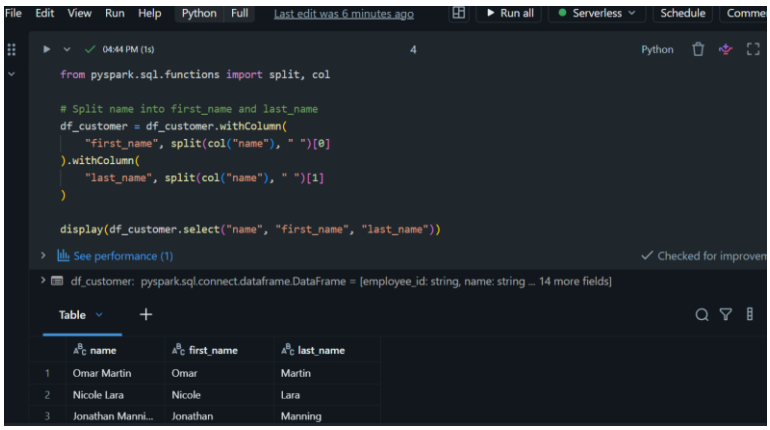
# Function to convert column names into snake_case
def to_snake_case(column_name):
    column_name = re.sub(r'([a-z0-9])([A-Z])', r'\1_\2', column_name)
    column_name = re.sub(r'\s+', '_', column_name)
    return column_name.lower()

# Rename all columns dynamically
for column in df_customer.columns:
    df_customer = df_customer.withColumnRenamed(
        column, to_snake_case(column)
    )

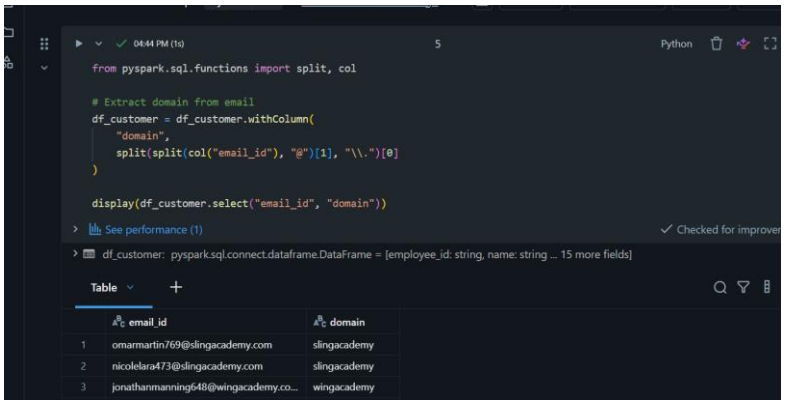
# Display updated column names
print(df_customer.columns)
```

```
['employee_id', 'name', 'email_id', 'address', 'gender', 'age', 'joining_date', 'registered', 'order_id', 'orders', 'spent',
'job', 'hobbies', 'is_married']
```

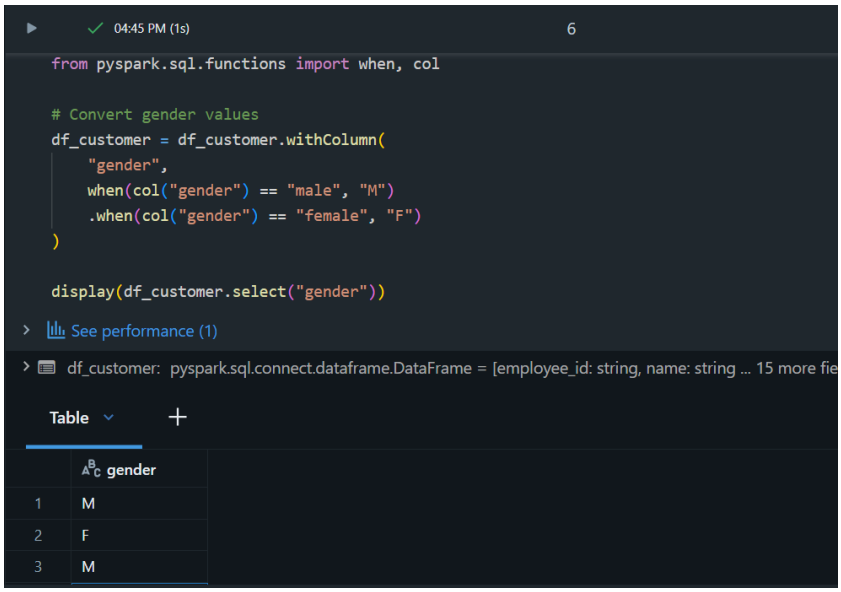
- By using the "Name" column split by " " and create two columns first_name and last_name



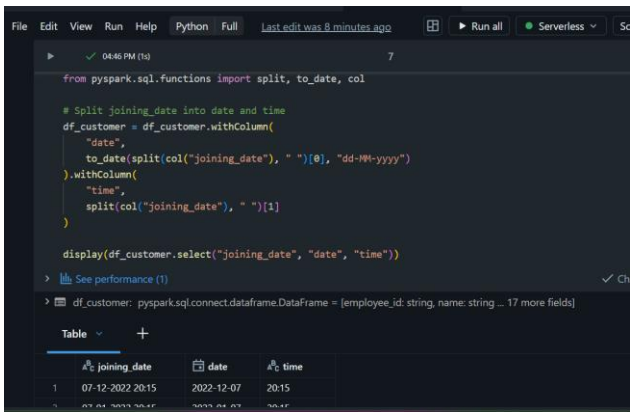
- Create column domain and extract from email columns Ex: Email = "josephrice131@slingacademy.com" domain="slingacademy"



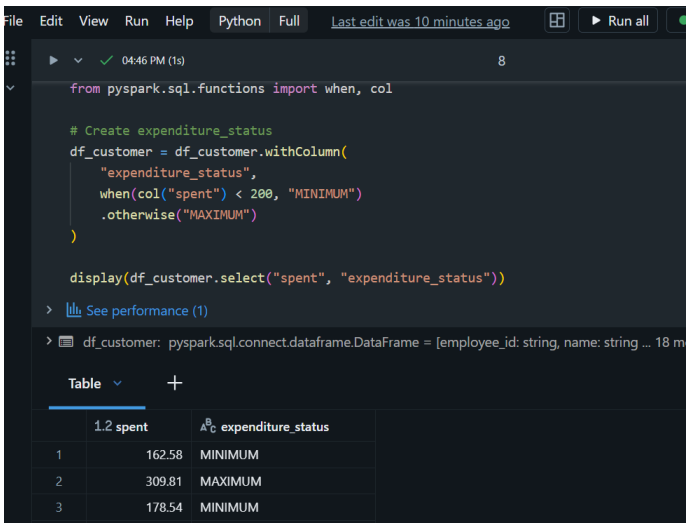
- Create a column ‘gender where male = "M" and Female="F"



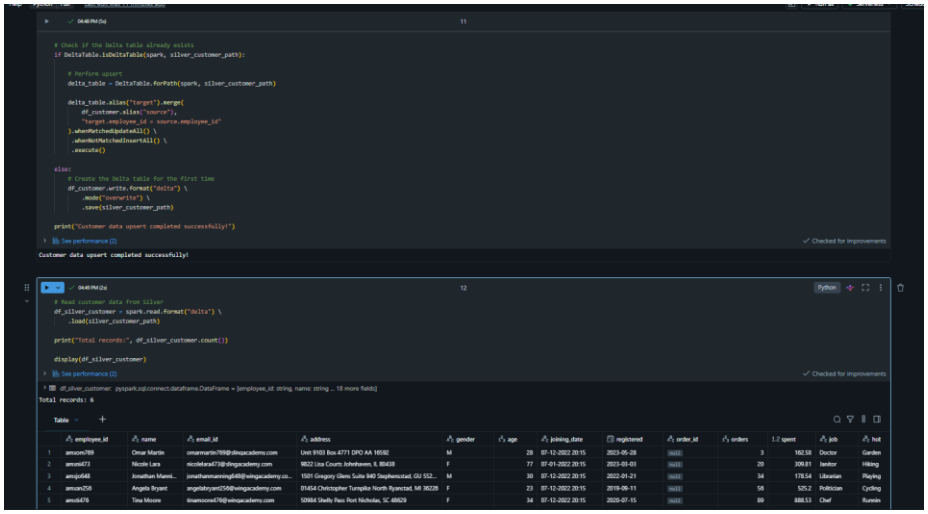
- From Joining date create two columns date and time by splitting based on " " delimiter.



Create a column expenditure-status, based on spent column is spent below 200 column value is "MINIMUM" else "MAXIMUM"



Write based on upsert [table_name: customer] (in silver layer path is silver/sales_view/tablename/{delta pearquet}



- 5. Product File
  All the column header should be in snake case in lower case (use same UDF Function)

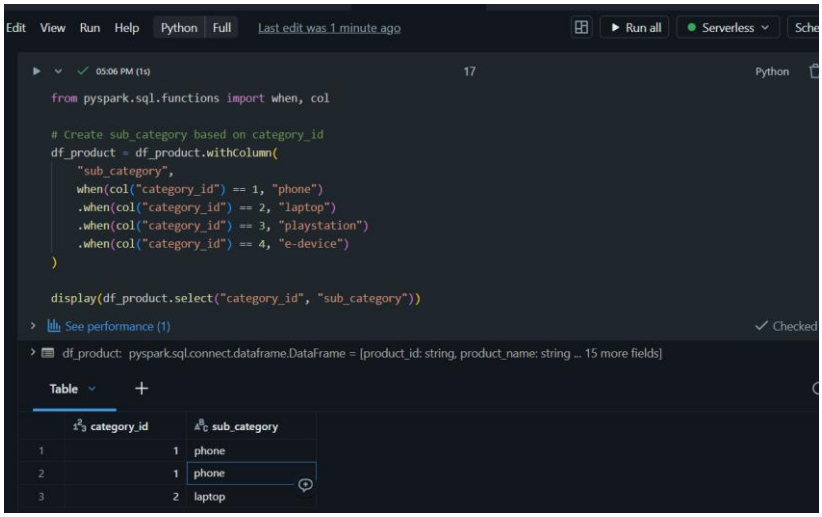


```
# Convert product column names to snake_case
for column in df_product.columns:
    df_product = df_product.withColumnRenamed(
        column, to_snake_case(column)
    )

# Display updated column names
print(df_product.columns)
```

```
['product_id', 'product_name', 'product_code', 'description', 'category_id', 'price', 'stock_quantity', 'supplier_id', 'created_at', 'updated_at', 'image_url', 'weight', 'expiry_date', 'is_active', 'tax_rate', 'store_id']
```

- Create a column sub_category (Use Category columns id category_id=1, "phone"; 2, "laptop"; 3,"playstation"; 4,"e-device"



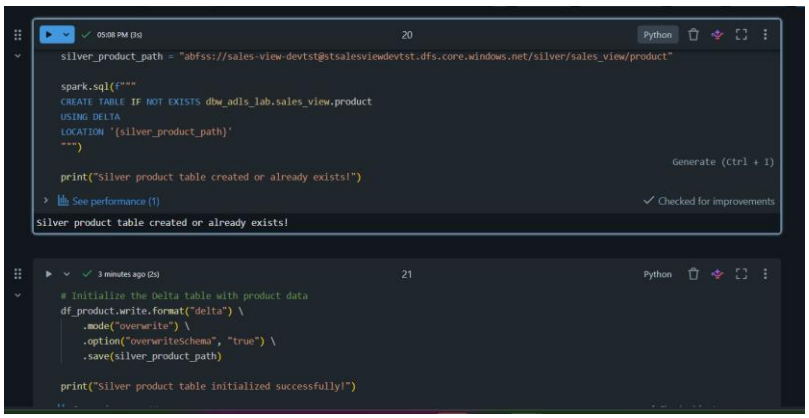
```
from pyspark.sql.functions import when, col

# Create sub_category based on category_id
df_product = df_product.withColumn(
    "sub_category",
    when(col("category_id") == 1, "phone")
    .when(col("category_id") == 2, "laptop")
    .when(col("category_id") == 3, "playstation")
    .when(col("category_id") == 4, "e-device")
)

display(df_product.select("category_id", "sub_category"))
```

	category_id	sub_category
1	1	phone
2	1	phone
3	2	laptop

- Write based on upsert [table_name: product](in silver layer path is silver/sales_view/tablename/{delta pearquet}



```
silver_product_path = "abfss://sales-view-devtst@stsalesviewdevtst.dfs.core.windows.net/silver/sales_view/product"

spark.sql(f"""
CREATE TABLE IF NOT EXISTS dbw_adls_lab.sales_view.product
USING DELTA
LOCATION '{silver_product_path}'
""")

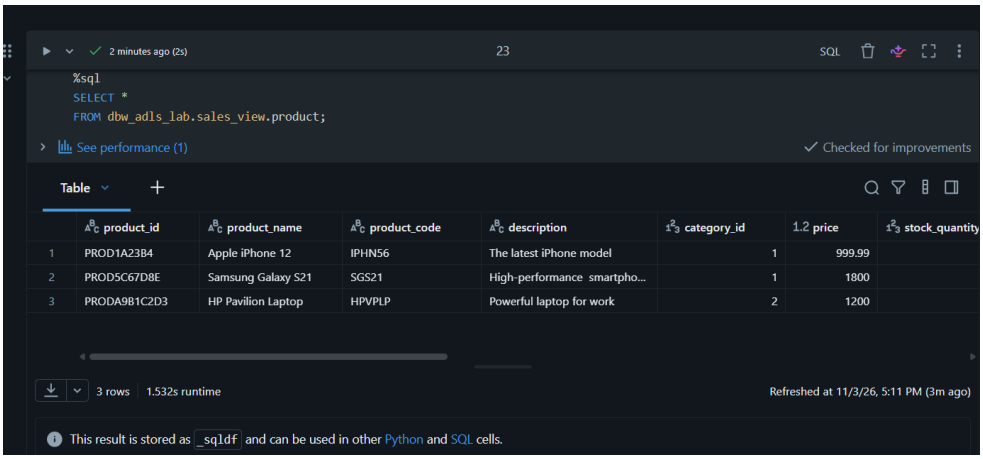
print("Silver product table created or already exists!")
```

```
Silver product table created or already exists!
```

```
# Initialize the Delta table with product data
df_product.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_product_path)

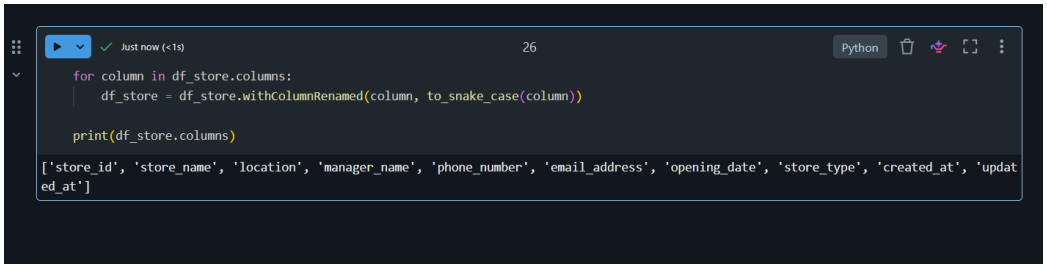
print("Silver product table initialized successfully!")
```


3 minutes ago (8s)
22
%sql
MERGE INTO dbw_adls_lab.sales_view.product AS target
USING product_updates AS source
ON target.product_id = source.product_id
WHEN MATCHED THEN
UPDATE SET *
WHEN NOT MATCHED THEN
INSERT *;
See performance (1)
Table
num_affected_rows
num_updated_rows
num_deleted_rows
num_inserted_rows
1
3
3
0
0


2 minutes ago (2s)
23
SQL
%sql
SELECT *
FROM dbw_adls_lab.sales_view.product;
See performance (1)
Checked for improvements
Table
product_id
product_name
product_code
description
category_id
price
stock_quantity
1 PROD1A23B4 Apple iPhone 12 IPHN56 The latest iPhone model 1 999.99
2 PROD5C67D8E Samsung Galaxy S21 SGS21 High-performance smartpho... 1 1800
3 PRODA9B1C2D3 HP Pavilion Laptop HPVPLP Powerful laptop for work 2 1200
3 rows
1.532s runtime
Refreshed at 11/3/26, 5:11 PM (3m ago)
This result is stored as _sqldf and can be used in other Python and SQL cells.

6. Store

- Read the data make sure header should be in snake case in lower case (use same UDF Function)



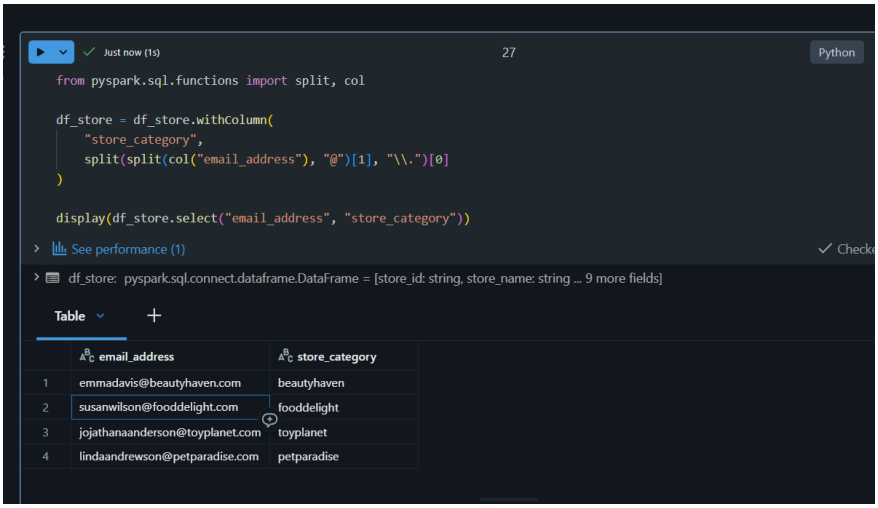
```
Just now (<1s)  26  Python

for column in df_store.columns:
    df_store = df_store.withColumnRenamed(column, to_snake_case(column))

print(df_store.columns)
```

```
['store_id', 'store_name', 'location', 'manager_name', 'phone_number', 'email_address', 'opening_date', 'store_type', 'created_at', 'updated_at']
```

- Create a store category columns and the value is exatracted from email Eg: "electromart" from johndoe@electromart.com



```
Just now (1s)  27  Python

from pyspark.sql.functions import split, col

df_store = df_store.withColumn(
    "store_category",
    split(split(col("email_address"), "@")[1], "\\.")[0]
)

display(df_store.select("email_address", "store_category"))
```

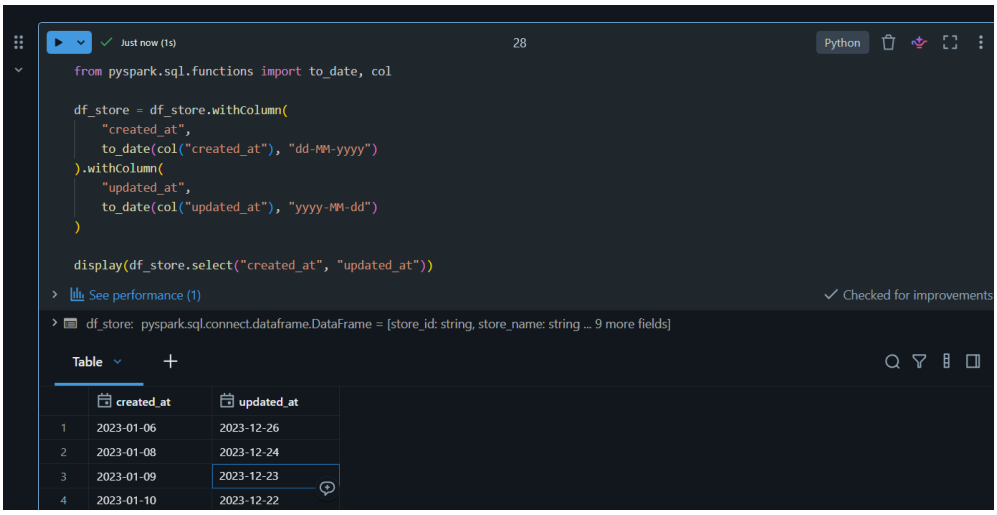
See performance (1) Checke

df_store: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_name: string ... 9 more fields]

Table +

	email_address	store_category
1	emmadavis@beautyhaven.com	beautyhaven
2	susanwilson@fooddelight.com	fooddelight
3	jojathanaanderson@toyplanet.com	toyplanet
4	lindaandrewson@petparadise.com	petparadise

- created_at, updated_at date as yyyy-MM-dd format



```
Just now (1s)  28  Python

from pyspark.sql.functions import to_date, col

df_store = df_store.withColumn(
    "created_at",
    to_date(col("created_at"), "dd-MM-yyyy")
).withColumn(
    "updated_at",
    to_date(col("updated_at"), "yyyy-MM-dd")
)

display(df_store.select("created_at", "updated_at"))
```

See performance (1) Checked for improvements

df_store: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_name: string ... 9 more fields]

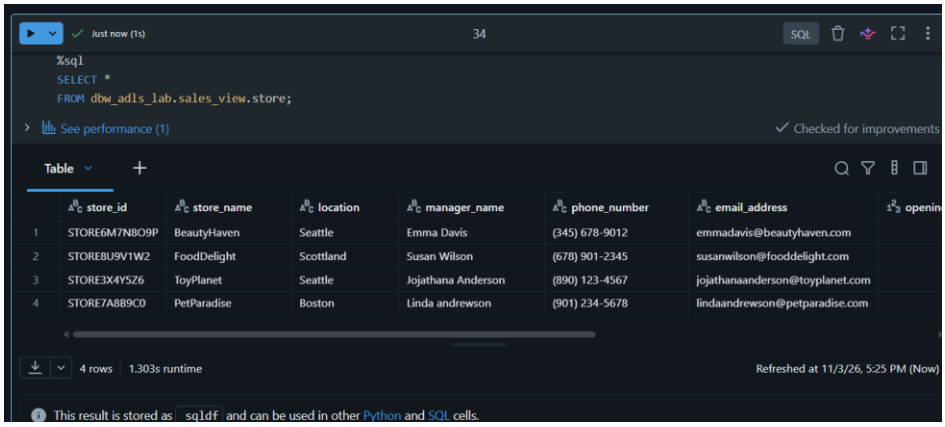
Table +

	created_at	updated_at
1	2023-01-06	2023-12-26
2	2023-01-08	2023-12-24
3	2023-01-09	2023-12-23
4	2023-01-10	2023-12-22

- Write based on upsert [table_name: store] (in silver layer path is silver/sales_view/tablename/{delta pearquet}

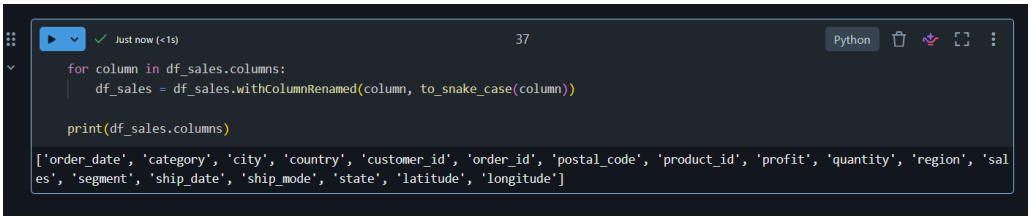






7. Sales

- Read the data make sure header shlould be in snake case in lower case (use same UDF Function)



```
for column in df_sales.columns:
    df_sales = df_sales.withColumnRenamed(column, to_snake_case(column))

print(df_sales.columns)
```

```
['order_date', 'category', 'city', 'country', 'customer_id', 'order_id', 'postal_code', 'product_id', 'profit', 'quantity', 'region', 'sales', 'segment', 'ship_date', 'ship_mode', 'state', 'latitude', 'longitude']
```

- Write based on upsert [table_name: customer_sales] (in silver layer path is silver/sales_view/tablename/{delta pearquet}



```
silver_sales_path = "abfss://sales-view-devtst@stsalesviewdevtst.dfs.core.windows.net/silver/sales_view/customer_sales"

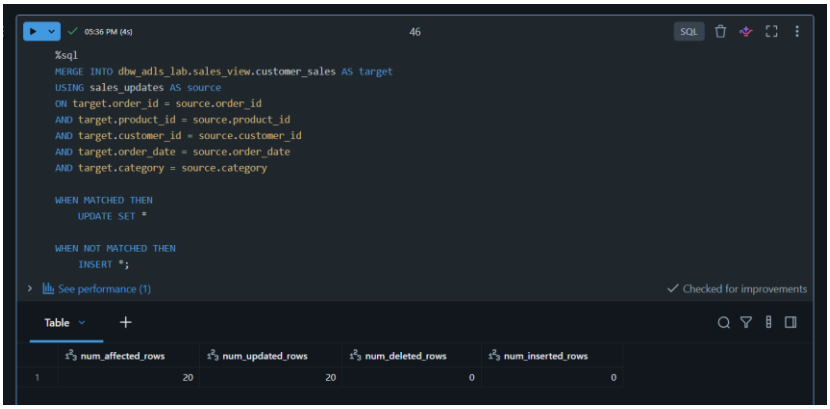
df_sales.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(silver_sales_path)

print("Silver Customer Sales data initialized successfully!")
```

```
Silver Customer Sales data initialized successfully!
```

```
%sql
CREATE TABLE IF NOT EXISTS dbw_adls_lab.sales_view.customer_sales
USING DELTA
LOCATION 'abfss://sales-view-devtst@stsalesviewdevtst.dfs.core.windows.net/silver/sales_view/customer_sales';
```

```
OK
```

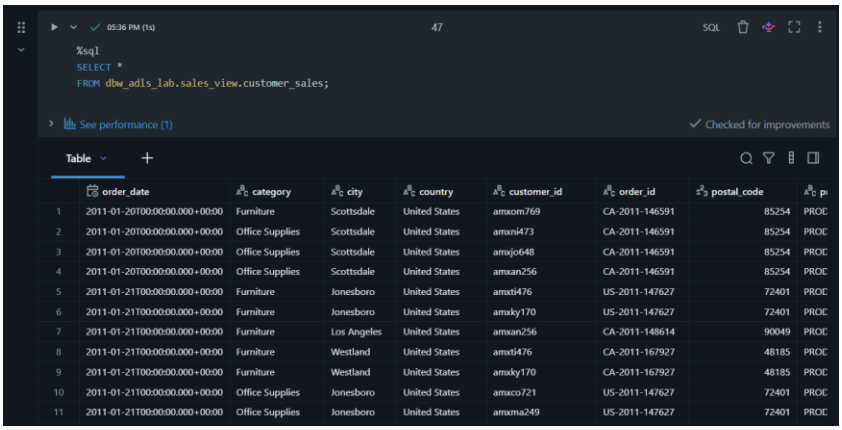


```
%sql
MERGE INTO dbw_adls_lab.sales_view.customer_sales AS target
USING sales_updates AS source
ON target.order_id = source.order_id
AND target.product_id = source.product_id
AND target.customer_id = source.customer_id
AND target.order_date = source.order_date
AND target.category = source.category

WHEN MATCHED THEN
    UPDATE SET *

WHEN NOT MATCHED THEN
    INSERT *;
```

	num_affected_rows	num_updated_rows	num_deleted_rows	num_inserted_rows
1	20	20	0	0



```
%sql
SELECT *
FROM dbw_adls_lab.sales_view.customer_sales;
```

	order_date	category	city	country	customer_id	order_id	postal_code	p
1	2011-01-20T00:00:00.000+00:00	Furniture	Scottsdale	United States	amxom769	CA-2011-146591	85254	PROD
2	2011-01-20T00:00:00.000+00:00	Office Supplies	Scottsdale	United States	amxni473	CA-2011-146591	85254	PROD
3	2011-01-20T00:00:00.000+00:00	Office Supplies	Scottsdale	United States	amxjo648	CA-2011-146591	85254	PROD
4	2011-01-20T00:00:00.000+00:00	Office Supplies	Scottsdale	United States	amxan256	CA-2011-146591	85254	PROD
5	2011-01-21T00:00:00.000+00:00	Furniture	Jonesboro	United States	amxti476	US-2011-147627	72401	PROD
6	2011-01-21T00:00:00.000+00:00	Furniture	Jonesboro	United States	amxky170	US-2011-147627	72401	PROD
7	2011-01-21T00:00:00.000+00:00	Furniture	Los Angeles	United States	amxan256	CA-2011-148614	90049	PROD
8	2011-01-21T00:00:00.000+00:00	Furniture	Westland	United States	amxti476	CA-2011-167927	48185	PROD
9	2011-01-21T00:00:00.000+00:00	Furniture	Westland	United States	amxky170	CA-2011-167927	48185	PROD
10	2011-01-21T00:00:00.000+00:00	Office Supplies	Jonesboro	United States	amxco721	US-2011-147627	72401	PROD
11	2011-01-21T00:00:00.000+00:00	Office Supplies	Jonesboro	United States	amxma249	US-2011-147627	72401	PROD

9. In gold layer

- using product and store table get the below data

  store_id,store_name,location,manager_name,product_name,product_code,description,category_id,price,stock_quantity,supplier_id,product_created_at,product_updated_at,image_url,weight,expiry_date,is_active,tax_rate.



```
df_product_store = df_product.join(
    df_store,
    on="store_id",
    how="left"
)

print("Product-Store records:", df_product_store.count())

display(df_product_store)
```

See performance (2) ✓ Checked for improvements

df_product_store: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, product_id: string ... 25 more fields]

Product-Store records: 3

Table

	store_id	product_id	product_name	product_code	description	category_id	price
1	ORE1D2E3F4G	PROD1A23B4	Apple iPhone 12	IPHN56	The latest iPhone model	1	999.99
2	ORE5H6I7J8K	PROD5C67D8E	Samsung Galaxy S21	SGS21	High-performance smartpho...	1	1800
3	OREL9M0N1O2	PRODA9B1C2D3	HP Pavilion Laptop	HPVPLP	Powerful laptop for work	2	1200



```
df_product_store_selected = df_product_store.select(
    df_product_store["store_id"],
    df_store["store_name"],
    df_store["location"],
    df_store["manager_name"],
    df_product["product_name"],
    df_product["product_code"],
    df_product["description"],
    df_product["category_id"],
    df_product["price"],
    df_product["stock_quantity"],
    df_product["supplier_id"],
    df_product["created_at"].alias("product_created_at"),
    df_product["updated_at"].alias("product_updated_at"),
    df_product["image_url"],
    df_product["weight"],
    df_product["expiry_date"],
    df_product["is_active"],
    df_product["tax_rate"]
)

display(df_product_store_selected)
```

- Read the delta table (using UDF functions)



```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType
from datetime import datetime

def get_year(date_value):
    if date_value is not None:
        return str(datetime.strptime(date_value, "%d-%m-%Y").year)
    return None

get_year_udf = udf(get_year, StringType())

df_product_store_delta = df_product_store_delta.withColumn(
    "product_created_year",
    get_year_udf("product_created_at")
)

display(df_product_store_delta.select(
    "product_name",
    "product_created_at",
    "product_created_year"
))
```

See performance (1) ✓ Checked for improvem

- Using the above data & customer_sales and get the below data

  OrderDate,Category,City,CustomerID,OrderID,Product ID, Profit,Region, Sales,Segment,ShipDate,ShipMode,latitude,longitude,store_name,location,manager_name,product_name,price,stock_quantity,image_url



```
df_product_store_delta = df_product_store_delta.join(
    df_product.select("product_id", "store_id"),
    on="store_id",
    how="left"
)

display(df_product_store_delta)
```

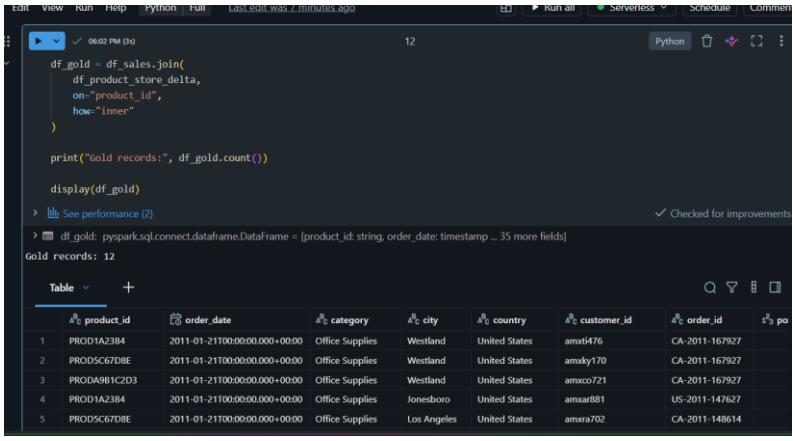
See performance (1) — Checked for improvements

df_product_store_delta: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_name: string ... 18 more fields]

Table

	store_id	store_name	location	manager_name	product_name	product_code	description
1	STORE1D2E3F4G	null	null	null	Apple iPhone 12	IPHN56	The latest iPhone mode
2	STORE5H6I7J8K	null	null	null	Samsung Galaxy S21	SGS21	High-performance sma
3	STOREL9M0N1O2	null	null	null	HP Pavilion Laptop	HPVPLP	Powerful laptop for wor

3 rows · 6.228s runtime — Refreshed at 11/3/26, 6:01 PM (14m ago)



```
df_gold = df_sales.join(
    df_product_store_delta,
    on="product_id",
    how="inner"
)

print("Gold records:", df_gold.count())

display(df_gold)
```

See performance (2) — Checked for improvements

df_gold: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, order_date: timestamp ... 35 more fields]

Gold records: 12

Table

	product_id	order_date	category	city	country	customer_id	order_id	po
1	PROD1A23B4	2011-01-21T00:00:00.000+00:00	Office Supplies	Westland	United States	amxti476	CA-2011-167927	
2	PROD5C67D8E	2011-01-21T00:00:00.000+00:00	Office Supplies	Westland	United States	amsky170	CA-2011-167927	
3	PRODA9B1C2D3	2011-01-21T00:00:00.000+00:00	Office Supplies	Westland	United States	amxco721	CA-2011-167927	
4	PROD1A23B4	2011-01-21T00:00:00.000+00:00	Office Supplies	Jonesboro	United States	amxar881	US-2011-147627	
5	PROD5C67D8E	2011-01-21T00:00:00.000+00:00	Office Supplies	Los Angeles	United States	amxra702	CA-2011-148614	



```
from pyspark.sql.functions import col

df_gold_final = df_gold.select(
    col("order_date").alias("OrderDate"),
    col("category").alias("Category"),
    col("city").alias("City"),
    col("customer_id").alias("CustomerID"),
    col("order_id").alias("OrderID"),
    col("product_id").alias("Product_ID"),
    col("profit").alias("Profit"),
    col("region").alias("Region"),
    col("sales").alias("Sales"),
    col("segment").alias("Segment"),
    col("ship_date").alias("ShipDate"),
    col("ship_mode").alias("ShipMode"),
    col("latitude"),
    col("longitude"),
    col("store_name"),
    col("location"),
    col("manager_name"),
    col("product_name"),
    col("price"),
    col("stock_quantity"),
    col("image_url")
```

- Write based on overwrite (table_name : StoreProductSalesAnalysis)(in gold layer path is gold/sales_view/tablename/{delta pearquet}



